

Laboratorio de Estructura de Computadores

Curso 2023-2024

Práctica 3: Programación en Ensamblador

Tutorial de uso del simulador del procesador RISC-V y ejemplo de programa en ensamblador

Durante este tutorial se explicará el funcionamiento del simulador del procesador RISC-V, que se puede descargar de *Moodle* (material de la unidad 3). El ejemplo propuesto realizará en ensamblador un programa que realice la misma función que el siguiente código C:

```
int a = 20;
int b = 10;
int c;
-----
int main()
{
    if(a < b)
        c = b;
    else c = a;
    while(1);
}
```

Ejercicio 1. Llamada a función y paso de parámetros

En este ejercicio se debe escribir en ensamblador un programa que tenga funcionalidad equivalente al programa en C propuesto y verificar su correcto funcionamiento. Para ello, debe entender cómo codificar una llamada a función en ensamblador y su paso de parámetros.

```
int X = 10;
int Y = 4;
int R;
-----
int calculaSumaMult (int a, int b)
{
    return(a+b) * 2;
}
-----
int main()
{
    R = calculaSumaMult(X,Y);
    while(1);
}
```

Tenga en cuenta que una única instrucción en el programa C, puede incluir múltiples operaciones en RISC-V:

- Dos lecturas de las variables X e Y de memoria a sendos registros.

- Escritura en la pila de las dos variables leídas (solo en el caso **1b**).
- Llamada a función.
- Recuperación del retorno guardado en pila (solo en el caso **1b**).
- Una escritura del retorno, desde registro en la variable R de memoria.

A su vez, la instrucción principal de la función calculaSumaMult incluye las siguientes operaciones:

- Recuperación de los parámetros leyendo la pila (solo en el caso **1b**).
- Suma de los parámetros.
- Multiplicación por 2 de la suma anterior.
- Escritura del resultado en pila (solo en el caso **1b**).
- Retorno a main.

Este ejercicio consta de dos partes claramente diferenciadas:

1a. El paso de parámetros entre la rutina principal y la función se realiza utilizando los registros específicos de RISC-V: a0-a7 para la llamada a la función y la devolución de resultados.

1b. El paso de parámetros entre la rutina principal y la función se realiza utilizando en ambos sentidos la pila del sistema.

Objetivo

Comprender el problema propuesto. Aprender a realizar llamadas a funciones y gestionar el paso de parámetros y retornos en sus dos mecanismos, usando registros específicos o utilizando la pila. plantear e implementar una solución para dicho problema y verificar su correcto funcionamiento. Este ejercicio se valorará en la calificación de evaluación continua si se termina y muestra al profesor durante la primera sesión de prácticas.

Ejercicio 2. Trabajar con vectores

En este ejercicio se debe escribir en ensamblador un programa que tenga funcionalidad equivalente al programa en C propuesto y verificar su correcto funcionamiento. Para ello, debe entender cómo codificar un bucle *for* en ensamblador y cómo realizar lecturas y escrituras de un vector.

```
#define N 10

int A[N]={2,2,4,6,5,6,7,8,9,10};
int B[N]={-1,-5,4,10,1,-2,5,10,-10,0};
int C[N];

-----

int main()
{
    int i;
    for(i=0; i<N; i++)
        C[i]=A[i]+B[i]*4;
    while(1);
}
```

Objetivo

Comprender el problema propuesto, el uso de bucles para operar con vectores, plantear e implementar una solución para dicho problema y verificar su correcto funcionamiento.

Nota: el valor de N hay que leerlo desde memoria (el define en lenguaje C, se convierte a una variable normal de memoria). Utilice las etiquetas A, B, C y N para estos cuatro elementos en memoria de datos. Este ejercicio se valorará en la calificación de evaluación continua si se termina y muestra al profesor durante la segunda sesión de prácticas.

Ejercicio 3. Funciones anidadas

Escriba el código ensamblador equivalente al programa proporcionado en lenguaje C. La rutina principal *main*, debe escribir en memoria el resultado final del código propuesto.

<pre>int A = 10; int B = 6; int R; int main () { R = funcion1 (A, B); while(1); }</pre>	<pre>int funcion1 (int i, int j) { k = i*4 - j*2; j = compl_a_2 (k); return j*2; }</pre>	<pre>int compl_a_2 (int i) { return 0 - i; }</pre>
--	--	--

Se sabe que el paso de parámetros en ambas rutinas se hace por registros. Se pide guardar en pila los registros estrictamente necesarios para el correcto funcionamiento del programa, pudiendo sobrescribir registros en la función anidada, siempre que sus funciones llamantes no los necesiten.

Nota: No olvide que, en una rutina anidada, la rutina que llama debe preservar y posteriormente recuperar el contenido del registro de enlace ra, utilizando en ambos casos la pila del sistema.

Objetivo

Comprender el problema propuesto: utilizar llamadas a funciones anidadas en donde hay que preservar entre otros el registro de enlace ra. Se trata de plantear e implementar una solución para dicho problema y verificar su correcto funcionamiento. Este ejercicio se valorará en la calificación de evaluación continua si se termina y muestra al profesor durante la segunda sesión de prácticas.

Ejercicio opcional. Desensamblar

En el ejercicio 1 se proporciona un programa (Desensamblar.txt) compatible con la arquitectura RISC-V descrita en las clases de teoría. En este ejercicio se debe desensamblar este programa, es decir, convertir las instrucciones en notación hexadecimal a notación ensamblador, para poder comprender la funcionalidad del programa.

El programa suministrado contiene dos segmentos, código y datos. Algunas instrucciones hacen referencia a posiciones del segmento de datos.

Objetivo

Desensamblar el programa propuesto y comprender su funcionamiento, para ello, se sugiere que con cada instrucción ensamblador, se incluya un comentario que explique la funcionalidad que genera. Comprobar con el simulador RARS de RISC-V, si el programa desensamblado coincide con el

programa en código máquina. Para ello se debe configurar la memoria como <<Compacta con dirección de inicio del segmento text en x0000>>. Para ello vaya a “**Settings -> Memory Configuration...**” y seleccione “**Compact, Text at Address 0**”. En toda esta práctica y a lo largo de este curso, debe repetir esta configuración siempre que use el simulador RARS, en concreto para todos los ejercicios y exámenes de esta práctica.

Este ejercicio **no** se valorará en la evaluación, sino que es opcional.